Output 4: In-Class Lab Guide

Lab 6: Textbook RSA vs. Secure RSA (OpenSSL & Python)

CAE-CD Knowledge Units: BCY-7, ACR-2, ACR-3, ACR-4 **Format:** Jupyter Notebook (`.ipynb` representation for VS Code on local VM)

Objective and Setup

In this lab, you will move beyond the theoretical mathematics of RSA to analyze applied implementations. You will use OpenSSL to examine key generation and ASN.1 structures. Then, utilizing Python's `cryptography` library, you will simulate the catastrophic failures of "Textbook RSA", specifically its multiplicative homomorphism. Finally, you will implement secure RSA-OAEP encryption and RSA-PSS digital signatures.

Prerequisites:

- Ubuntu VM (Local or Cyber Range)
- VS Code with Jupyter Notebook extension installed
- Python `cryptography` library

In [21]:
```
# Run this cell to ensure dependencies are installed
!pip install cryptography --quiet
import os
import math
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization
print("Dependencies loaded successfully.")
```

```
[notice] A new release of pip is available: 23.2.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
Dependencies loaded successfully.
```

Part 1: Key Generation and Inspection via OpenSSL

We will utilize the OpenSSL CLI to generate a 2048-bit RSA key pair and inspect its underlying mathematical components, managing PEM formats and examining ASN.1 structures. *Constraint Check (ACR-1, BCY-7): Understanding the elements and structure of the asymmetric cryptosystem.*

In [22]:
```
%%bash
# 1. Generate a 2048-bit private key
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits

# 2. Extract the public key from the private key
openssl rsa -pubout -in private_key.pem -out public_key.pem

# 3. Inspect the raw mathematical components of the private key (ASN.1 struc
openssl rsa -text -noout -in private_key.pem | head -n 25
```

```
...+.....+.......+.........+..+...+....+++++++++++++++++++++++++++++++++++++++
++*.......+.....+.........+....+.....+++++++++++++++++++++++++++++++++++++++++
*.........+......+.+...+.....+....+............+.....+...+.......+...
+............+...............+........+............+...+......+.......+.....
+.......+...+...+...........+......+..........+...+.....
+......................+.+........+.+.....+............
+...........................+...+...............+...............+...
+....................+....+...........+....+......+.....+...+....+..+...
+..........+............+.........+..+....+...+.....+.+...........++++++
..+.........+......+.+.....+....+++++++++++++++++++++++++++++++++++++++*...
+..+.+............+...........+....+..+.+..+.+...............+..
+.............+..+...+......+.+...+............+..+.+.....+......
+...............+.+............+.....+...+.........+...+++++++++++++++++++++
++++++++++++++++++*.+..........+......+..............+......+....+...+......
+..+.......+...............+...+....................+...+.+...+.....+.......
+...+......+.........+..+.+..............+...+......+.+...+...
+................+............+.+..+...+.......................+.....
+.......+..+.+..+...+.......+........+............+......+......+..........
+.....+...+.+.....+......+...+.......+.....+...................+......+...
+...........+............+.........+.+..+.......+...........+...+.+......
+.....+....+...+...............+..+.+...+..............+.+.....+....+...
+..............+...+.+.....+.+.....+.+..+...+....++++++
writing RSA key
```

```
Private-Key: (2048 bit, 2 primes)
modulus:
    00:89:22:ff:6b:1a:15:a1:4c:8a:64:60:5a:2d:f2:
    03:d5:ad:1a:75:10:e4:29:36:5e:7f:b6:82:31:9d:
    a0:00:91:a6:a0:37:15:9a:ca:ee:d6:e1:8c:ec:a1:
    9e:f2:59:4f:53:65:63:62:1f:ff:76:24:e1:37:c5:
    12:3e:6c:ce:34:bf:dc:f6:ab:7c:dd:a5:f3:96:91:
    b4:f5:83:74:02:3c:fd:17:6a:56:80:ec:e8:21:08:
    8c:ee:9f:f0:40:7a:7f:25:2e:0d:40:9e:74:7f:05:
    30:53:a0:3f:28:38:dd:13:80:31:f3:95:ba:55:cf:
    1a:ab:5f:b3:d8:d4:10:12:01:79:85:51:80:06:b8:
    d8:83:6c:bd:c2:22:48:39:f2:59:3c:87:fc:8a:00:
    91:cf:d6:5b:35:52:b7:b3:b3:ff:b5:41:f7:f0:44:
    60:19:54:80:79:a6:18:da:07:a7:a4:ef:99:50:77:
    53:ea:fb:9b:1c:8d:4b:82:8a:8e:e0:8d:5c:bc:76:
    9d:37:6a:c3:03:9e:c7:67:48:a0:f6:6f:99:a6:14:
    e7:50:04:c3:6b:b1:ee:ac:b7:b8:da:16:4e:6c:cb:
    e9:a9:7f:07:62:cd:c4:56:f4:1e:f2:91:05:e5:f4:
    de:b4:6c:47:50:14:b1:e4:47:10:13:22:7c:f3:03:
    12:7d
publicExponent: 65537 (0x10001)
privateExponent:
    03:cc:fd:71:18:f6:30:a6:52:33:e0:69:2f:6c:f2:
    59:42:c9:4a:25:74:0e:86:59:df:fd:44:4a:e7:e1:
    89:b7:11:fc:17:c3:e1:bc:c0:87:b4:82:c0:4b:e5:
d5:ad:1a:75:10:e4:29:36:5e:7f:b6:82:31:9d:
    a0:00:91:a6:a0:37:15:9a:ca:ee:d6:e1:8c:ec:a1:
    9e:f2:59:4f:53:65:63:62:1f:ff:76:24:e1:37:c5:
    12:3e:6c:ce:34:bf:dc:f6:ab:7c:dd:a5:f3:96:91:
    b4:f5:83:74:02:3c:fd:17:6a:56:80:ec:e8:21:08:
    8c:ee:9f:f0:40:7a:7f:25:2e:0d:40:9e:74:7f:05:
    30:53:a0:3f:28:38:dd:13:80:31:f3:95:ba:55:cf:
    1a:ab:5f:b3:d8:d4:10:12:01:79:85:51:80:06:b8:
    d8:83:6c:bd:c2:22:48:39:f2:59:3c:87:fc:8a:00:
    91:cf:d6:5b:35:52:b7:b3:b3:ff:b5:41:f7:f0:44:
    60:19:54:80:79:a6:18:da:07:a7:a4:ef:99:50:77:
    53:ea:fb:9b:1c:8d:4b:82:8a:8e:e0:8d:5c:bc:76:
    9d:37:6a:c3:03:9e:c7:67:48:a0:f6:6f:99:a6:14:
    e7:50:04:c3:6b:b1:ee:ac:b7:b8:da:16:4e:6c:cb:
    e9:a9:7f:07:62:cd:c4:56:f4:1e:f2:91:05:e5:f4:
    de:b4:6c:47:50:14:b1:e4:47:10:13:22:7c:f3:03:
    12:7d
publicExponent: 65537 (0x10001)
privateExponent:
    03:cc:fd:71:18:f6:30:a6:52:33:e0:69:2f:6c:f2:
    59:42:c9:4a:25:74:0e:86:59:df:fd:44:4a:e7:e1:
    89:b7:11:fc:17:c3:e1:bc:c0:87:b4:82:c0:4b:e5:
```

Lab Question 1: Look at the output above. You will see modulus, publicExponent, and privateExponent. What integer is utilized for the publicExponent by default in OpenSSL, and why is this value chosen for computational efficiency? *(Reference your asynchronous material on the standard choice in industry and the square-and-multiply algorithm)*.

Lab Answer 1: The default OpenSSL public exponent is 65537 (hex 0x10001). This is chosen because it is a Fermat prime with very low bit density, so square-and-multiple is fast (mostly squaring, little multiplying). This number is still big enough to avoid weaknesses that small exponents like 3 have. It provides a strong security/performance balance.

Part 2: The Failure of Textbook RSA

"Textbook RSA" (c = m^e mod n) has a mathematical property that an attacker can exploit: it is multiplicatively homomorphic. In this section, you will simulate a scenario where an attacker intercepts an encrypted bid and manipulates the underlying plaintext without decrypting it. *Constraint Check (ACR-2, ACR-4): Evaluate a cryptosystem and explain its vulnerability to attacks.*

In [23]:
```
# We will use small textbook RSA parameters for this demonstration.
# Do NOT use textbook RSA in production.
p = 61
q = 53
n = p * q
phi_n = (p - 1) * (q - 1)
e = 17 # Public Exponent
# Calculate private exponent d (Extended Euclidean Algorithm)
d = pow(e, -1, phi_n)

# Scenario: Bob encrypts a bid of $1,000,000
# We will use 1000000 to represent the bid
original_bid = 1000000
ciphertext = pow(original_bid, e, n)
print(f"Original Bid: ${original_bid}")
print(f"Intercepted Ciphertext: {ciphertext}")

# Attacker Intercepts the ciphertext and manipulates it.
# Attacker wants to multiply the bid by 2. They compute r^e mod n.
multiplier = 2
tamper_factor = pow(multiplier, e, n)

# Attacker manipulates the ciphertext: c' = c * r^e mod n
tampered_ciphertext = (ciphertext * tamper_factor) % n
print(f"Tampered Ciphertext sent to Server: {tampered_ciphertext}")

# Server decrypts the tampered ciphertext: m' = (c')^d mod n
decrypted_tampered_bid = pow(tampered_ciphertext, d, n)
print(f"Server Decrypted Bid: ${decrypted_tampered_bid}")
```

```
Original Bid: $1000000
Intercepted Ciphertext: 1528
Tampered Ciphertext sent to Server: 132
Server Decrypted Bid: $2006
```

Lab Question 2: In the code above, the attacker changed the decrypted message to m * r without knowing m or the private key d. Explain how the mathematical properties of modular exponentiation allow this to occur.

Lab Answer 2: The attacker only exploits the modular exponentiation rules to transform the ciphertext so the decrypted plaintext is multiplied by (r) (mod (n)).

More in-depth answer (with AI assistance): As described in the above comments, RSA is multiplicatively homomorphic under modulo (n).

If the original ciphertext is (c = m^e mod n), an attacker can choose any factor (r) and compute c' = c * r^e mod n Then during decryption: (c')^d === (m^e * r^e)^d * (mr)^{ed} === mr mod n since (ed === 1 mod phi(n)). So the attacker never needs (m) or (d)

Part 3: Secure Implementation with RSA-OAEP

To stop the attacks demonstrated in Part 2, production systems use Optimal Asymmetric Encryption Padding (OAEP). OAEP uses a random seed to make encryption probabilistic and breaks the multiplicative homomorphism by using a Feistel-like structure. *Constraint Check (ACR-3): Explain the application of cryptography in secure applications.*

In [24]:
```
# Generate a secure 2048-bit keypair using the cryptography library
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
public_key = private_key.public_key()

message = b"CONFIDENTIAL: Troop movements at 0500."

# Configure OAEP Padding
oaep_padding = padding.OAEP(
    mgf=padding.MGF1(algorithm=hashes.SHA256()),
    algorithm=hashes.SHA256(),
    label=None
)

# Encrypt the same message twice to demonstrate probabilistic nature
ciphertext_1 = public_key.encrypt(message, oaep_padding)
ciphertext_2 = public_key.encrypt(message, oaep_padding)

print(f"Ciphertext 1 (First 32 bytes): {ciphertext_1[:32].hex()}")
print(f"Ciphertext 2 (First 32 bytes): {ciphertext_2[:32].hex()}")
print(f"Are ciphertexts identical? {ciphertext_1 == ciphertext_2}") # Always

# Decrypt to verify
```

```
recovered_message = private_key.decrypt(ciphertext_1, oaep_padding)
print(f"\nRecovered Message: {recovered_message.decode('utf-8')}")
```

```
Ciphertext 1 (First 32 bytes): 2b75d6fe06a878a941e0f67a78421704eeea0be7d7480
02bf53c7ea0739d1a58
Ciphertext 2 (First 32 bytes): 34981d8940f2284c218fedc5738dc999d00e6606b0236
a156fd60eb9231b5bc3
Are ciphertexts identical? False

Recovered Message: CONFIDENTIAL: Troop movements at 0500.
```

Part 4: Digital Signatures with RSA-PSS

RSA is not just an encryption algorithm—it is also a digital signature scheme. Digital signatures provide authentication, integrity, and non-repudiation. Here, we will sign a financial authorization utilizing the modern standard, RSA-PSS.

In [25]:

```
# Configure PSS Padding
pss_padding = padding.PSS(
    mgf=padding.MGF1(hashes.SHA256()),
    salt_length=padding.PSS.MAX_LENGTH
)

# Sign a message
document = b"I authorize the transfer of $10,000 to Account 7742."

# The Private Key holder signs the document
signature = private_key.sign(document, pss_padding, hashes.SHA256())
print(f"Signature generated. Length: {len(signature)} bytes.")

# A third party verifies the document with the Public Key
try:
    public_key.verify(signature, document, pss_padding, hashes.SHA256())
    print("\n[SUCCESS] Signature is VALID. Document authentic.")
except Exception as e:
    print("\n[ERROR] Signature is INVALID!")

# Demonstrate tamper detection: modify the message
tampered_document = b"I authorize the transfer of $99,000 to Account 7742."
print("\nAttempting to verify tampered document...")
try:
    public_key.verify(signature, tampered_document, pss_padding, hashes.SHA2
    print("[SUCCESS] Tampered signature is VALID.")
except Exception as e:
    print("[ERROR] Tampered signature is INVALID. Tampering detected!") # Ex
```

```
Signature generated. Length: 256 bytes.

[SUCCESS] Signature is VALID. Document authentic.

Attempting to verify tampered document...
[ERROR] Tampered signature is INVALID. Tampering detected!
```

Lab Submission: Export this executed Jupyter Notebook as a PDF. Ensure all code blocks have been run, output is visible, and Lab Questions 1 and 2 are answered in added markdown cells. Upload the PDF to the iLearn drop box.